GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport
IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es
GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL
Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 05.2. FLEXBOX

LM

Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es
2024/2025
Versión:250107.2028

Licencia

Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Introducción.....5
2. Conceptos.....5
2.1 Elementos.....5
2.2 Modalidades de flex.....6
2.3 Dirección de los ejes.....7
2.4 Contenedor flex multilínea.....9
2.4.1 Atajo: Dirección de los ejes.....11
2.5 Huecos (gaps).....12
2.5.1 Atajo: Huecos.....12
3. Alinear y centrar con Flex CSS.....13
3.1 Introducción.....13
3.2 Propiedades de alineación.....13
3.3 Alineación de elementos.....14
3.4 La propiedad justify-content.....14
3.5 La propiedad align-items.....16
3.6 Alineación multilínea.....17
3.6.1 La propiedad align-content.....17
3.7 Alineaciones específicas.....22
3.7.1 La propiedad align-self.....22
3.7.2 Atajo: Alineaciones.....24
3.8 Orden de los elementos.....25
4. Elementos flexibles en CSS.....26
4.1 Introducción.....26
4.2 Propiedades de flexibilidad.....26
4.2.1 La propiedad flex-basis.....26
4.2.2 La propiedad flex-grow.....27
4.2.3 La propiedad flex-shrink.....29
4.2.4 Atajo: La propiedad flex.....30
4.3 Elementos flex animados.....31
4.4 Transiciones.....32
4.4.1 Transición de entrada y salida.....33
4.4.2 Transiciones por propiedad.....33
5. Bibliografía.....34

UD05.2. FLEXBOX

1. INTRODUCCIÓN

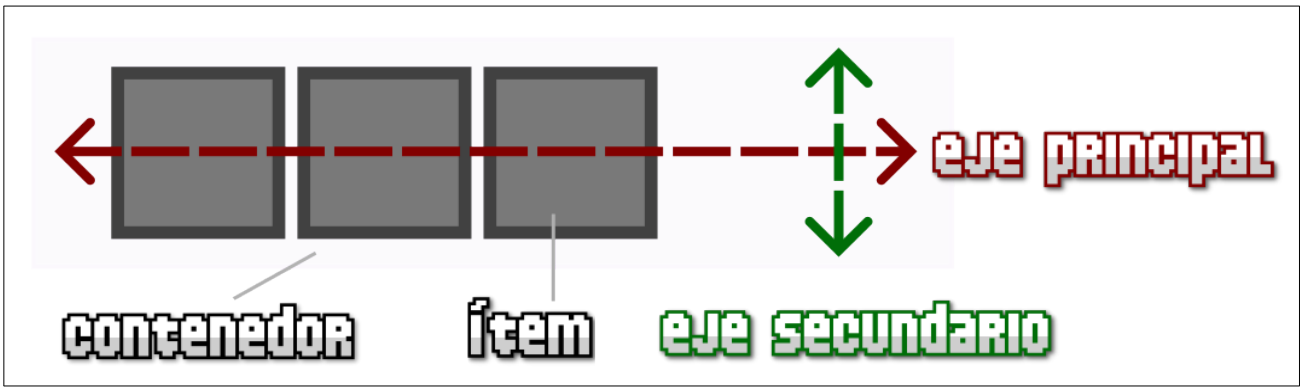
En CSS, inicialmente se utilizaba el posicionamiento (*static, relative, absolute...*), los elementos en línea o en bloque (*y derivados*) o la propiedad **float** para realizar maquetaciones, lo que a grandes rasgos no dejaba de ser un sistema de creación de diseños bastante tosco que no encajaba con los retos que tenemos en la actualidad: sistemas de escritorio, dispositivos móviles, múltiples resoluciones, etc...

Flex (*también llamado flexbox*) es un sistema de **elementos flexibles** que llega con la idea de olvidar estos mecanismos y acostumbrarnos a una mecánica más potente, limpia y personalizable, en la que los elementos HTML se adaptan y colocan automáticamente y es más fácil personalizar los diseños de una página web.

2. CONCEPTOS

2.1 Elementos

Para empezar a utilizar **flex** lo primero que debemos hacer es conocer algunos de los elementos básicos de este nuevo esquema, que son los siguientes:



- **Contenedor**: Es el elemento padre que tendrá en su interior cada uno de los ítems flexibles. Observa que al contrario que muchas otras estructuras CSS, por norma general, en Flex establecemos las propiedades al elemento padre.
    - **Eje principal**: Los contenedores flexibles tendrán una orientación principal específica. Por defecto, el eje principal del contenedor flex es en **horizontal** (*en fila*).
    - **Eje secundario**: De la misma forma, los contenedores flexibles tendrán una orientación secundaria, perpendicular a la principal. Si la principal es en

horizontal, la secundaria será en **vertical** (*y viceversa*).

- **Ítem**: Cada uno de los **hijos** que tendrá el contenedor en su interior.

2.2 Modalidades de flex

Una vez tenemos claro esto, imaginemos el siguiente escenario:

```
<div class="container"> <!-- Flex container -->
 <div class="item item-1">1</div> <!-- Flex items -->
 <div class="item item-2">2</div>
 <div class="item item-3">3</div>
</div>

.container {
   display: flex;
   flex-direction: row;
   background: gray;
   border: 6px solid black;
 }
 .item {
   display: flex;
   justify-content: center;
   align-items: center;
   width: 70px;
   height: 70px;
   margin: 10px;
   background: darkgray;
   font-size: 2em;
   color: white;
   border: black solid 2px;
 }
```

Para activar el modo **flex**, utilizaremos sobre el elemento contenedor la propiedad display, y especificaremos el valor `flex` o `inline-flex` (*dependiendo de como queramos que se comporte el contenedor*):

Tipo de elemento	Descripción
`inline-flex`	Establece un contenedor en línea, similar a `inline-block` (ocupa solo el contenido).
`flex`	Establece un contenedor en bloque, similar a `block` (ocupa todo el ancho del padre).

Por defecto, y sólo con esto, observaremos que los elementos se disponen todos sobre una misma línea. Esto ocurre porque estamos utilizando el modo **flex** y estaremos trabajando con ítems flexibles básicos, garantizando que no se desbordarán ni mostrarán los problemas que, por ejemplo, tienen los porcentajes sobre elementos que no utilizan flex.

2.3 Dirección de los ejes

Existen dos propiedades principales para manipular la dirección y comportamiento de los ítems a lo largo del eje principal del contenedor. Son las que veremos a continuación:

Propiedad	Valor	Significado
`flex-direction`	**row** | row-reverse | column | column-reverse	Cambia la orientación del eje principal.

Mediante la propiedad `flex-direction` podemos modificar la dirección del **eje principal** del contenedor para que se oriente en horizontal (*valor por defecto*) o en vertical. Además, también podemos incluir el sufijo `-reverse` para indicar que coloque los ítems en orden inverso.

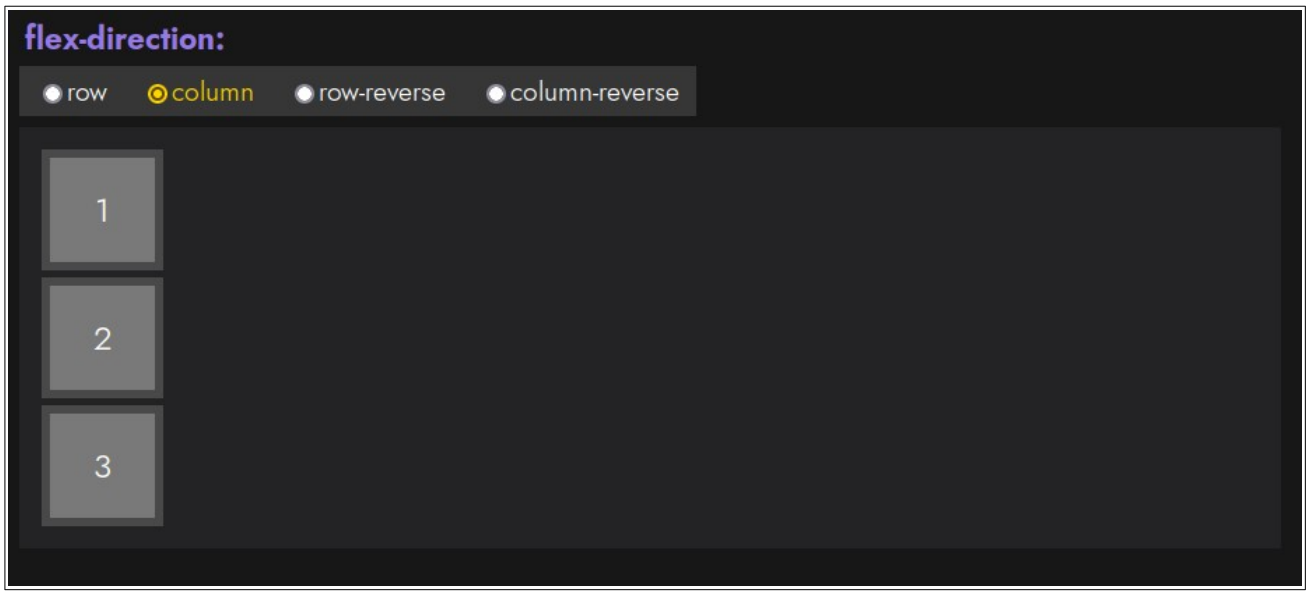
Valor	Descripción
`row`	Establece la dirección del eje principal en **horizontal**.
`row-reverse`	Establece la dirección del eje principal en **horizontal invertido**.
`column`	Establece la dirección del eje principal en **vertical**.

Valor	Descripción
`column-reverse`	Establece la dirección del eje principal en **vertical invertido**.

Esto nos permite tener un control muy alto sobre el orden de los elementos en una página.

Veamos la aplicación de estas propiedades sobre el ejemplo anterior, para modificar el flujo del eje principal del contenedor:

```
.container {
  display: flex;
  flex-direction: column;
  background: steelblue;
}
.item {
  background: grey;
}
```



2.4 Contenedor flex multilínea

En general, **flex** se suele utilizar para estructuras de una sola dimensión, es decir, contenedores que sólo van en una dirección. Sin embargo, existe una propiedad denominada `flex-wrap` con la que podemos especificar un comportamiento especial del contenedor.

Por defecto, si un elemento no cabe dentro de nuestro contenedor flex, los elementos se harán más pequeños (*son flexibles*) para ajustarlos al contenedor. Este es el comportamiento por defecto de un contenedor **flex**. Sin embargo, con la propiedad `flex-wrap` podemos cambiar este comportamiento y permitir que nuestro contenedor flex se desborde, convirtiéndose en un **contenedor flex multilínea**.

Propiedad	Valor	Significado
`flex-wrap`	**nowrap** | wrap | wrap-reverse	Evita o permite el desbordamiento (multilínea).

Los valores que puede tomar esta propiedad, son las siguientes:

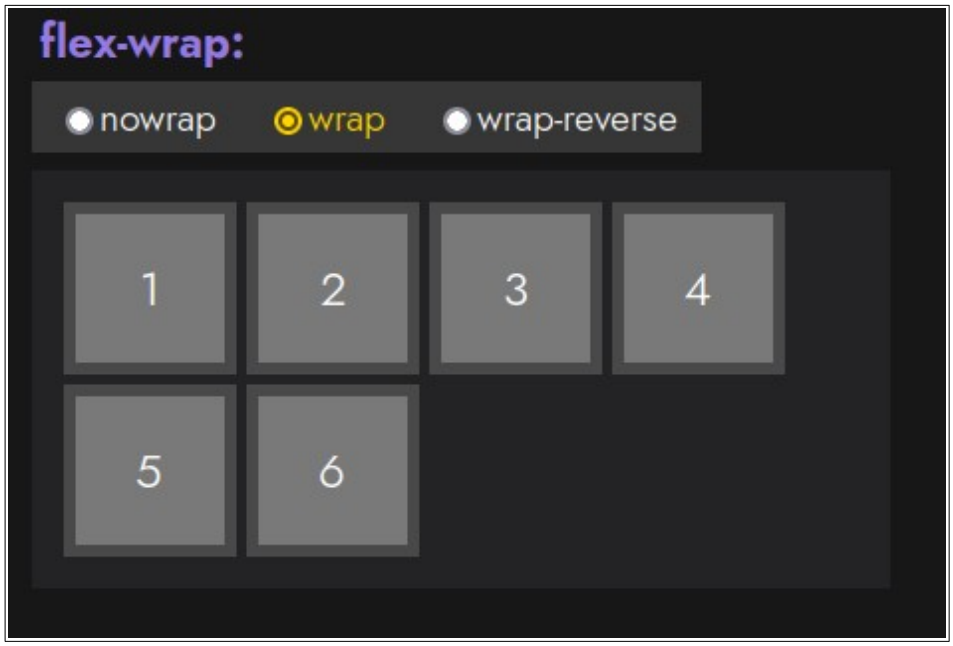
Valor	Descripción
nowrap	Los ítems se ajustan para ocupar el tamaño del contenedor (no permite desbordamiento en múltiples líneas).
wrap	Establece los ítems en modo multilínea (permite que se desborde el contenedor).
wrap-reverse	Establece los ítems en modo multilínea, pero en dirección inversa.

Teniendo en cuenta estos valores de la propiedad `flex-wrap`, podemos conseguir cosas como la siguiente:

```
.container {
  display: flex;
  flex-wrap: wrap;
  background: steelblue;
  width: 200px;
}
.item {
  background: grey;
  width: 50%;
}
```

En el caso de especificar **nowrap** (*u omitir la propiedad `flex-wrap`*) en el contenedor, los 3 ítems se mostrarían en una misma línea del contenedor (*que es el comportamiento por defecto*). En ese caso, si tenemos 3 ítems, cada uno debería tener aproximadamente `66px` de ancho. Un tamaño de `100px` por ítem, sumaría un total de `300px`, que no cabrían en el contenedor de `200px`, por lo que **flex** reajusta los ítems flexibles para que quepan todos en la misma línea, manteniendo las mismas proporciones.

Sin embargo, si especificamos `wrap` en la propiedad `flex-wrap`, lo que permitimos es que el contenedor se pueda desbordar, pasando a ser un contenedor **multilínea**, que mostraría los **ítems** que quepan en la primera linea y el resto en las líneas siguientes. El valor `wrap-reverse` haría exactamente lo mismo, pero con el orden inverso.



2.4.1 Atajo: Dirección de los ejes

Recuerda que existe una propiedad de atajo (short-hand) llamada `flex-flow`, con la que podemos resumir los valores de las propiedades `flex-direction` y `flex-wrap`, especificándolas en una sola propiedad y ahorrándonos utilizar las propiedades concretas:

```
.container {
 /* flex-flow: <flex-direction> <flex-wrap>; */
```

```
  flex-flow: row wrap;
}
```

2.5 Huecos (gaps)

Existen dos propiedades de flexbox que han surgido recientemente: `row-gap` y `column-gap`. Dichas propiedades, permiten establecer el tamaño de un «hueco» entre ítems desde el elemento padre contenedor, y que eliminan la necesidad de estar utilizando `padding` o `margin` en los elementos hijos, con las complicaciones que ello suele llevar:

Propiedad	Valor	Descripción
`row-gap`	**normal** | size	Espacio entre filas (sólo funciona con `flex-direction: column`)
`column-gap`	**Normal** | size	Espacio entre columnas (sólo funciona con `flex-direction: row`)

Ten en cuenta que, como **flex** es un sistema para diseños de una sola dimensión, sólo una de las dos propiedades tendrá efecto. Si la propiedad `flex-direction` está establecida en `column`, podrás utilizar `row-gap`, y en el caso de que la propiedad `flex-direction` se encuentre en `row`, podrás utilizar el `column-gap`.

Eso sí, es posible usar ambas si tenemos la propiedad `flex-wrap` definida a `wrap` y, por lo tanto, disponemos de multicolumnas flexbox, ya que en este caso si podemos separar elementos por filas y por columnas.

> Ten en cuenta que los huecos **sólo se aplican entre elementos**, y no entre un elemento hijo y su contenedor padre.

2.5.1 Atajo: Huecos

En **Flex CSS** existe una propiedad de atajo para los huecos, denominada `gap`. Con esta propiedad podemos indicar de una sola vez valores para las propiedades `row-gap` y `column-gap`, de forma que escribimos menos y es más cómodo en ciertas situaciones:

Propiedad	Valor	Descripción
`gap`	**0** | size	Aplica el tamaño indicado para el hueco en ambos ejes.
`gap`	**0 0** | size size	Aplica los tamaños indicados para el hueco del eje X y el eje Y.

Como se puede ver, por defecto, el tamaño de los huecos es de **0**, sin embargo, podemos utilizar tanto las propiedades individuales como la propiedad de atajo gap para modificar estos tamaños.

A continuación, podemos ver un ejemplo de su utilización en este fragmento de código:

```
.container {
  /* 2 parámetros: <row> <column> */
  gap: 4px 8px;
  /* Equivalente a */
  row-gap: 4px;
  column-gap: 8px;

  /* 1 parámetro: usa el mismo para ambos */
  gap: 4px;
  /* Equivalente a */
  row-gap: 4px;
  column-gap: 4px;
}
```

3. ALINEAR Y CENTRAR CON FLEX CSS

3.1 Introducción

Cuestiones habituales en el mundo de CSS suelen ser **«Cómo centrar con Flex»**, **«cómo alinear verticalmente»** o **«cómo alinear horizontalmente»**. Vamos a dar un repaso a las propiedades de alineación de elementos, para que no se te vuelva a resistir como distribuir los elementos de un contenedor **Flex**.

3.2 Propiedades de alineación

Ahora tenemos un control básico de un contenedor con ítems flexibles. Pero para alinear correctamente, necesitamos conocer las propiedades existentes dentro de flex para disponer los ítems dependiendo de nuestro objetivo.

Vamos a echar un vistazo a las siguientes propiedades, donde algunas actúan en el **eje principal** 1️⃣ (*recordemos que por defecto es el horizontal*), mientras que otras actúan en el **eje secundario** 2️⃣ (*por defecto, el eje vertical*):

Propiedad	Valor	Actúa en eje
`justify-content`	**start** | end | center | space-between | space-around | space-evenly	1️⃣
`align-items`	start | end | center | **stretch** | baseline	2️⃣
`align-content`	start | end | center | space-between | space-around | space-evenly | **stretch**	2️⃣

De esta pequeña lista, hay que centrarse en la primera y la segunda propiedad, ya que son las principales. La última propiedad, `align-content` solo tiene efecto si tenemos un contenedor **flex multilínea**.

3.3 Alineación de elementos

- `justify-content`: Se utiliza para alinear los ítems del **eje principal** (*por defecto, el horizontal*).
- `align-items`: Usada para alinear los ítems del **eje secundario** (*por defecto, el vertical*).
- `align-content`: Se utiliza para alinear el contenido del **eje secundario** entre líneas (*sólo en contenedor multilínea*).

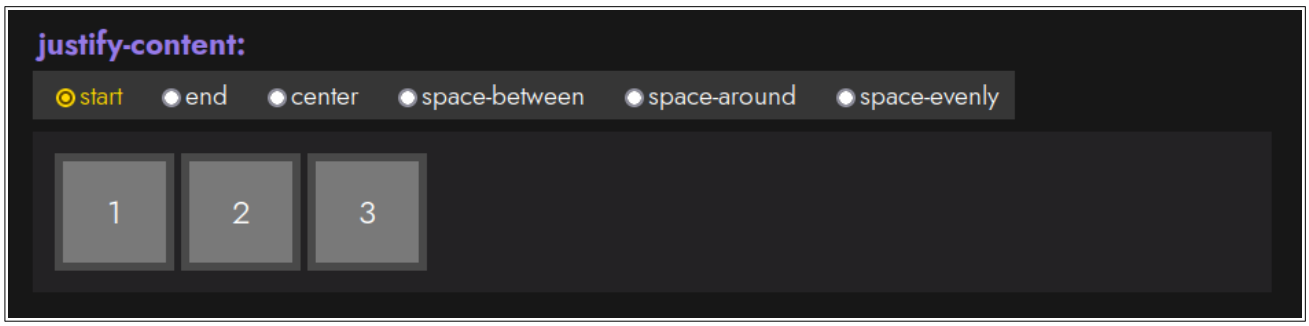
3.4 La propiedad justify-content

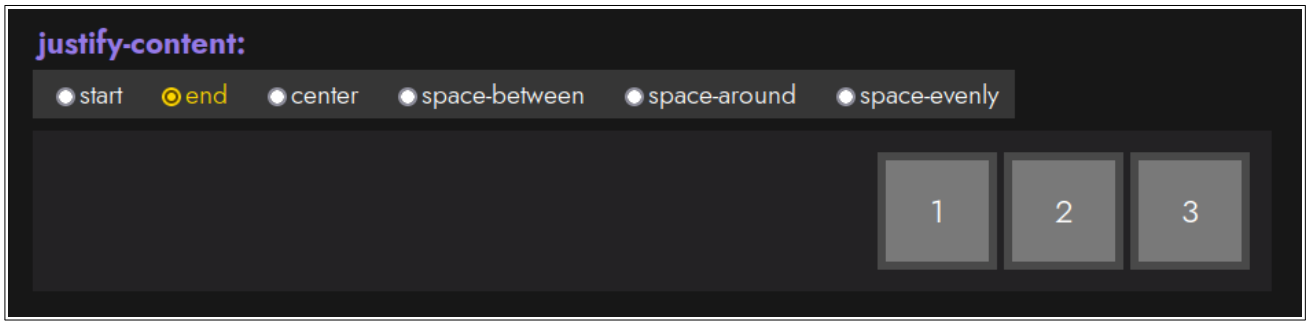
La primera propiedad, `justify-content`, sirve para colocar los ítems de un contenedor mediante una disposición concreta a lo largo del **eje principal** (*por defecto, en horizontal*). Los valores que puede tomar esta propiedad son los siguientes:

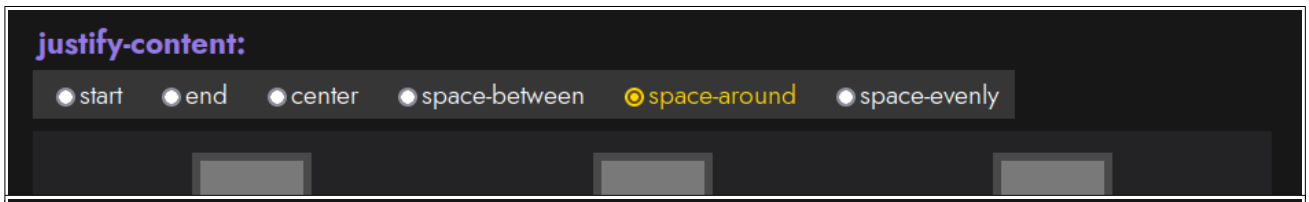
Valor	Descripción
start	Agrupa los ítems al **inicio** del eje principal.
`end`	Agrupa los ítems al **final** del eje principal.
`center`	Agrupa los ítems al **centro** del eje principal.
`space-between`	Distribuye los ítems dejando espacio **entre ellos**.
`space-around`	Distribuye los ítems dejando espacio **alrededor de ellos**.
`space-evenly`	Distribuye como `space-around`, pero con un **espacio exactamente igual** alrededor de ellos.

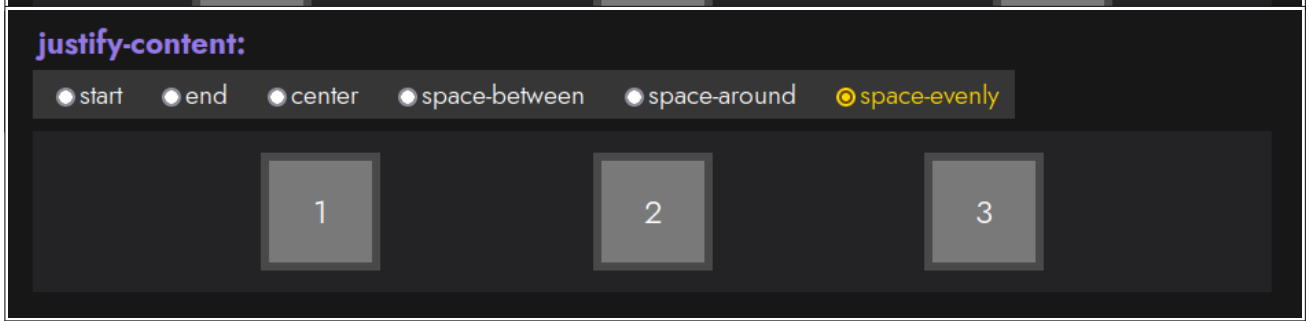
Con cada uno de estos valores, modificaremos la disposición de los ítems del contenedor donde se aplica, pasando a colocarse como se ve en el ejemplo interactivo

siguiente (*nótense los números para observar el orden de cada ítem*):









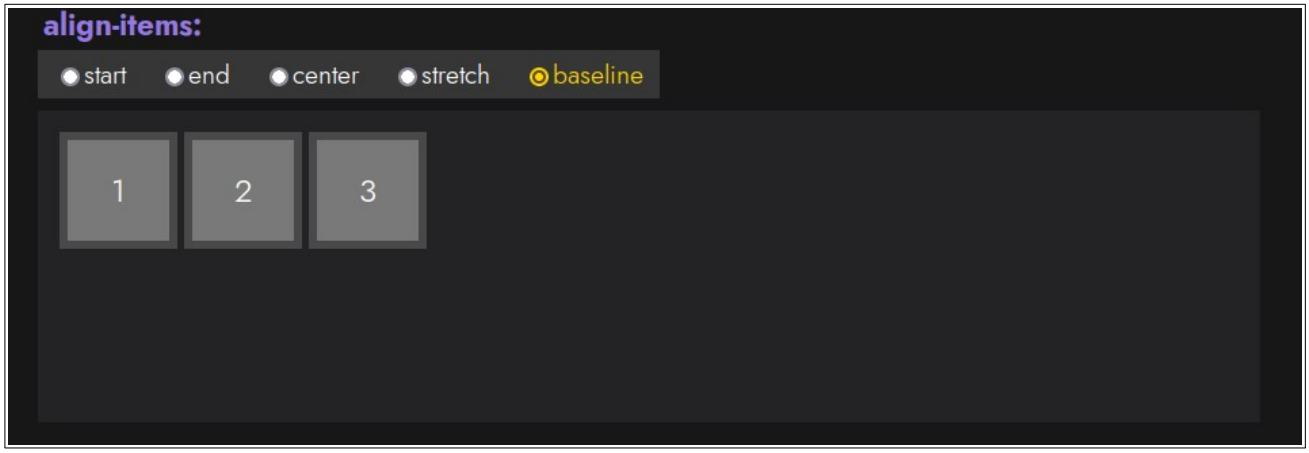
3.5 La propiedad align-items

Existe otra propiedad importante denominada `align-items`. Se encarga de alinear los ítems en el **eje secundario** del contenedor. Hay que tener cuidado de no confundir `align-items` con `align-content`, puesto que el segundo actúa sobre cada una de las líneas de un contenedor multilinea (*no tiene efecto si no usamos `flex-wrap`*), mientras que `align-items` lo hace sobre la única línea que tiene un contenedor flex sin wrap.

Los valores que puede tomar `align-items` son los siguientes:

Valor	Descripción
`start`	Alinea los ítems al **inicio** del eje secundario.
`end`	Alinea los ítems al **final** del eje secundario.
`center`	Alinea los ítems al **centro** del eje secundario.
stretch	Alinea los ítems **estirándolos** de modo que cubran desde el inicio hasta el final del contenedor.
`baseline`	Alinea los ítems en el contenedor según la **base** del contenido de los ítems del contenedor.

Veamos un ejemplo con `align-items`:



3.6 Alineación multilínea

Una vez entendidos los casos anteriores, debemos atender a la propiedad `align-content`, que es un caso particular de `align-items`. Nos servirá cuando estemos tratando con un **contenedor flex multilinea** creado mediante `flex-wrap`. Los contenedores multilinea son un tipo de contenedor en el que, cuando los ítems no caben en el ancho disponible, el eje principal se divide en múltiples líneas.

3.6.1 La propiedad `align-content`

De esta forma, `align-content` servirá para alinear cada una de las líneas del contenedor multilínea. Los valores que puede tomar la propiedad `align-content` son los siguientes:

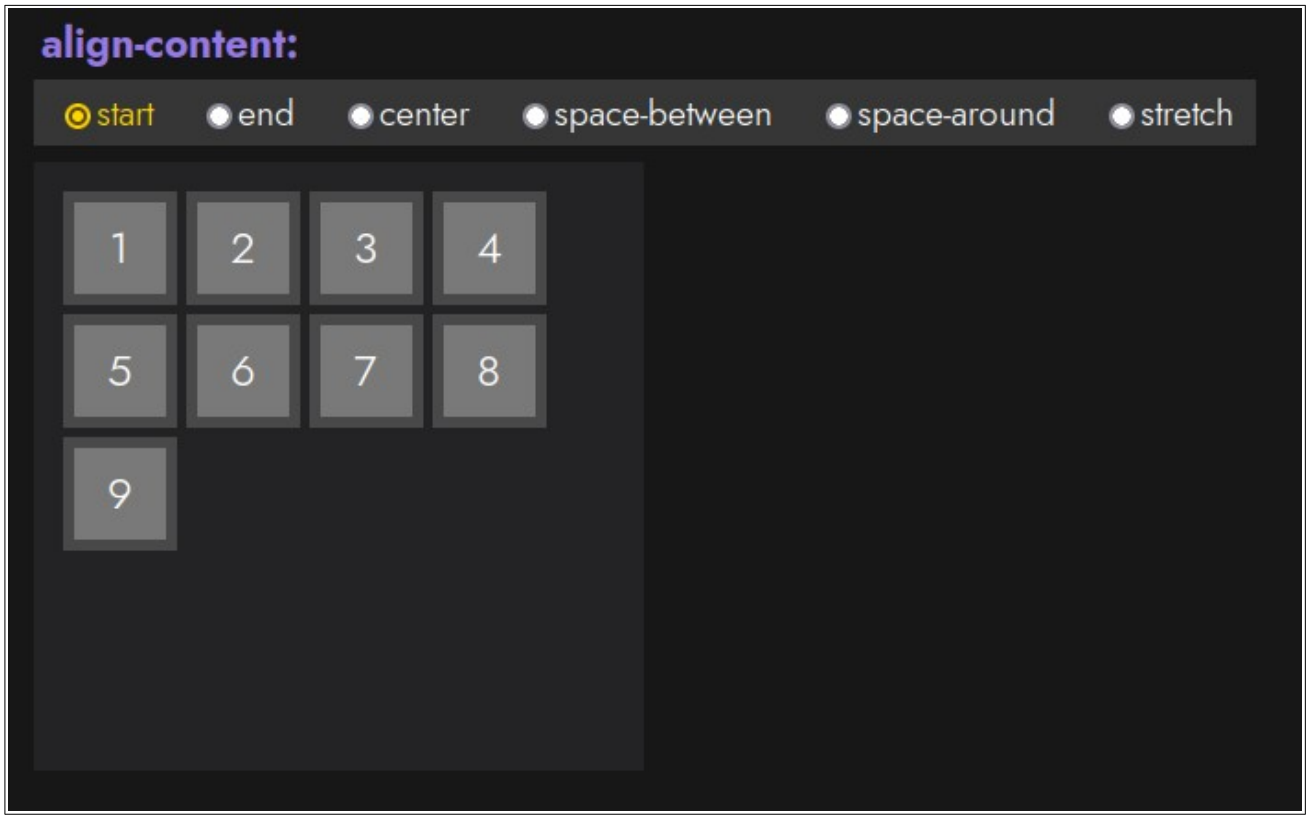
Valor	Descripción
`start`	Agrupa los ítems al **inicio** del eje principal.

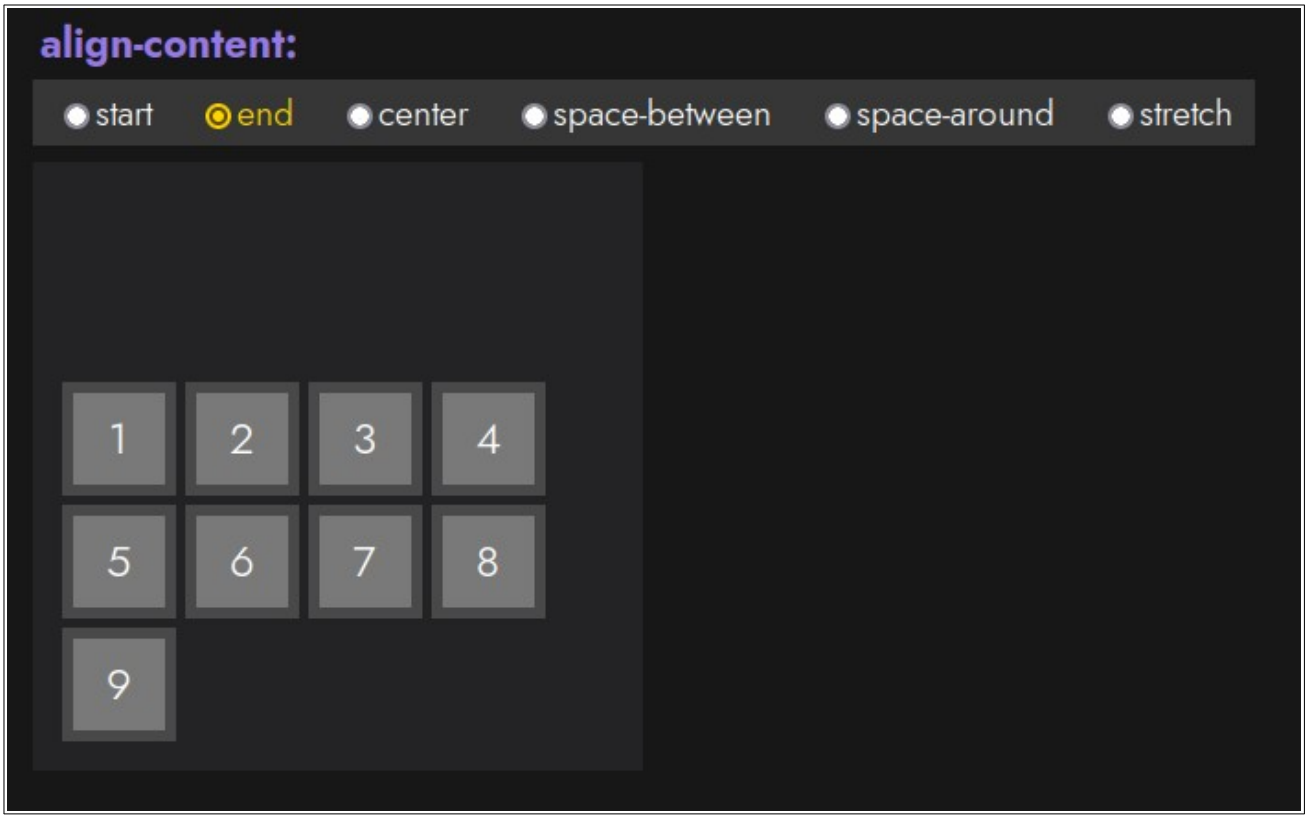
Valor	Descripción
`end`	Agrupa los ítems al **final** del eje principal.
`center`	Agrupa los ítems al **centro** del eje principal.
`space-between`	Distribuye los ítems desde el **inicio hasta el final**.
`space-around`	Distribuye los ítems dejando el mismo espacio **a los lados** de cada uno.
stretch	**Estira** los ítems para ocupar de forma equitativa todo el espacio.

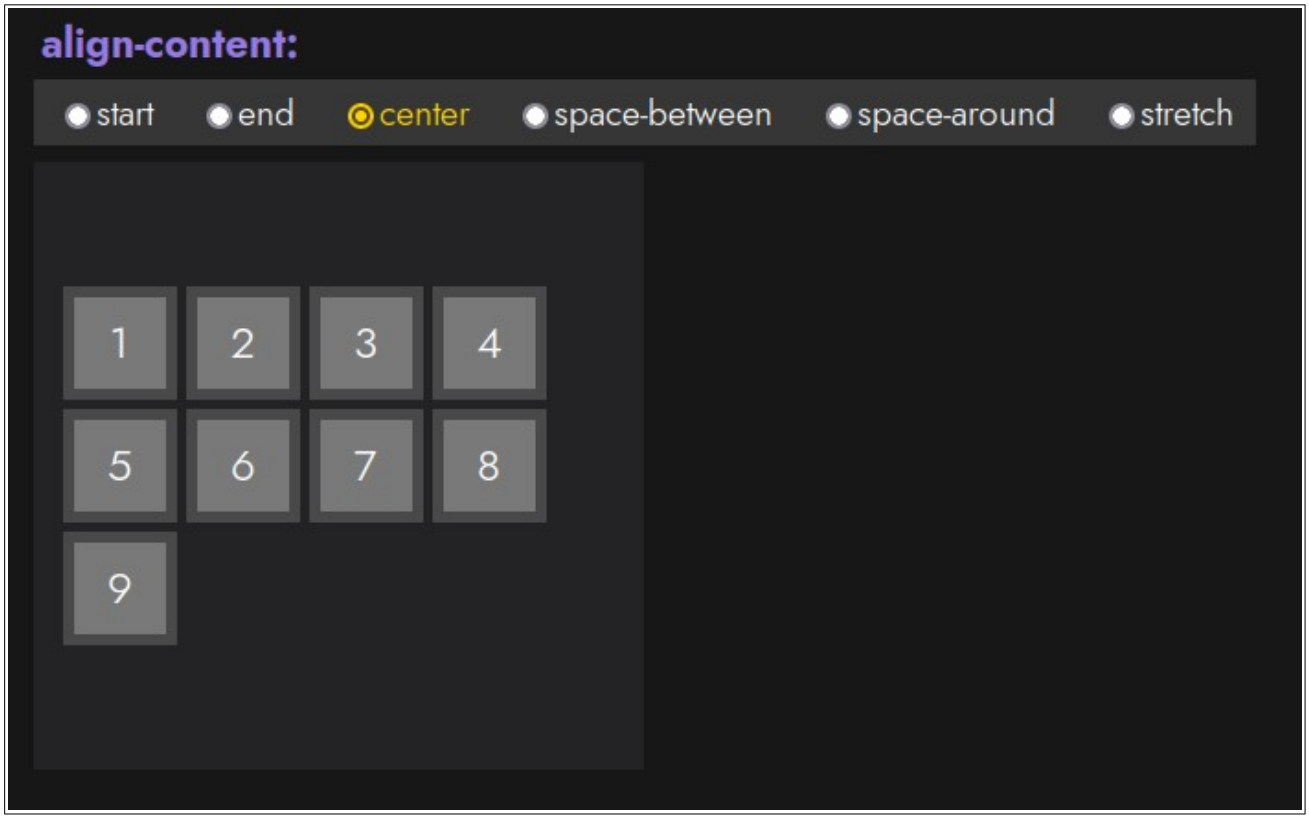
Recuerda que estaremos modificando la disposición en vertical, salvo que estemos usando `flex-direction: column`, en cuyo caso, el eje principal sería horizontal.

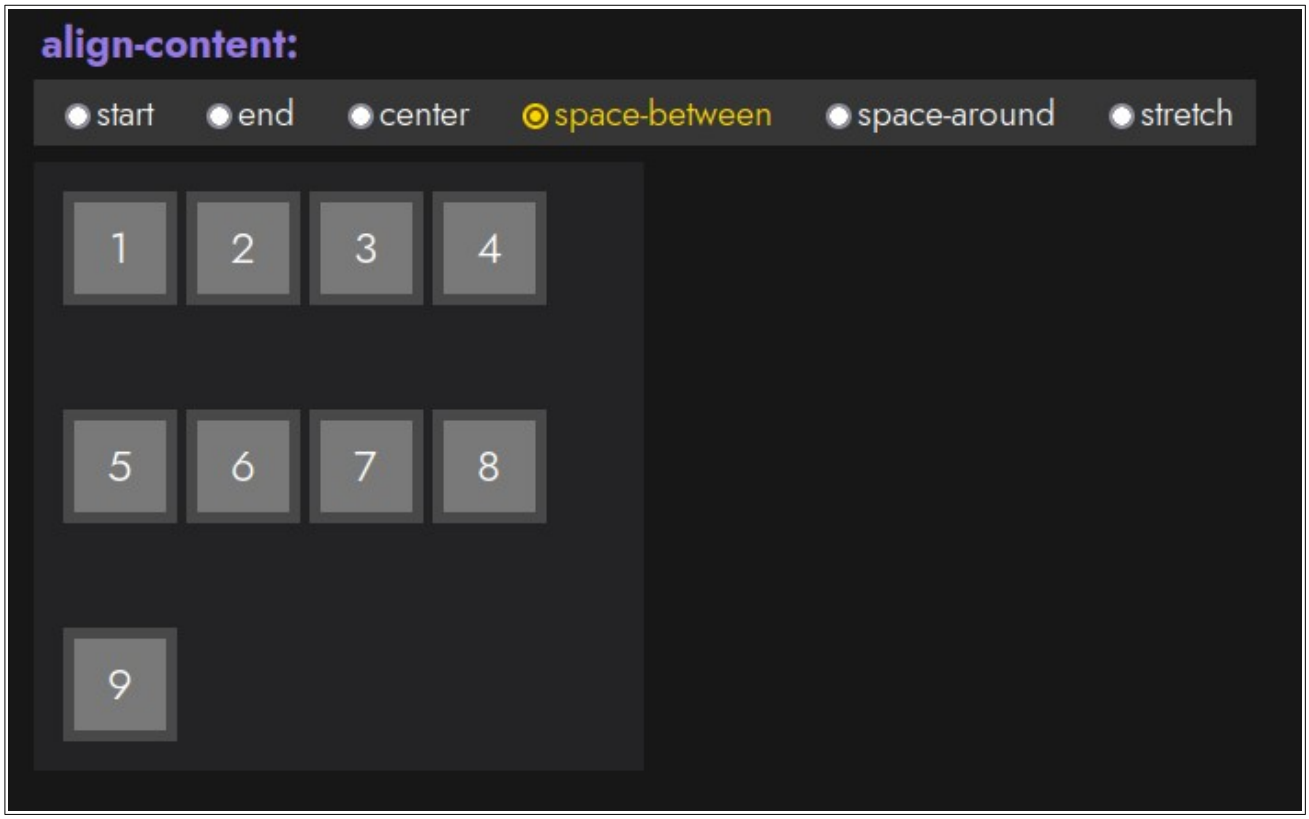
En el ejemplo siguiente, veremos que al indicar un contenedor de **200 píxels de alto** con ítems de **50px** de alto y un **flex-wrap** establecido para tener contenedores multilinea, podemos utilizar la propiedad `align-content` para alinear los ítems de forma vertical de modo que se queden en la zona inferior del contenedor:

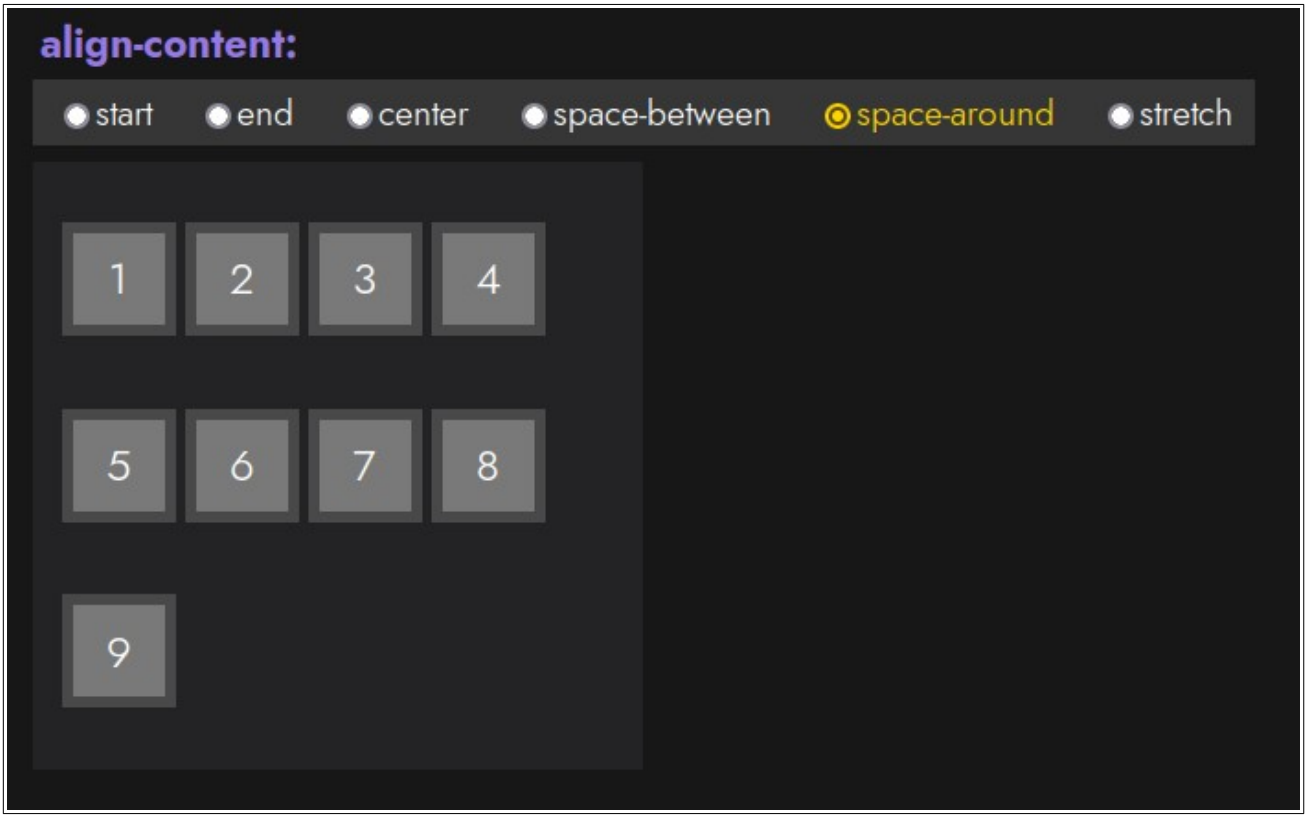
```
.container {
  background: #CCC;
  display: flex;
  width: 200px;
  height: 200px;

  flex-wrap: wrap;
  align-content: end;
}

.item {
  background: #777;
  width: 50%;
  height: 50px;
}
```

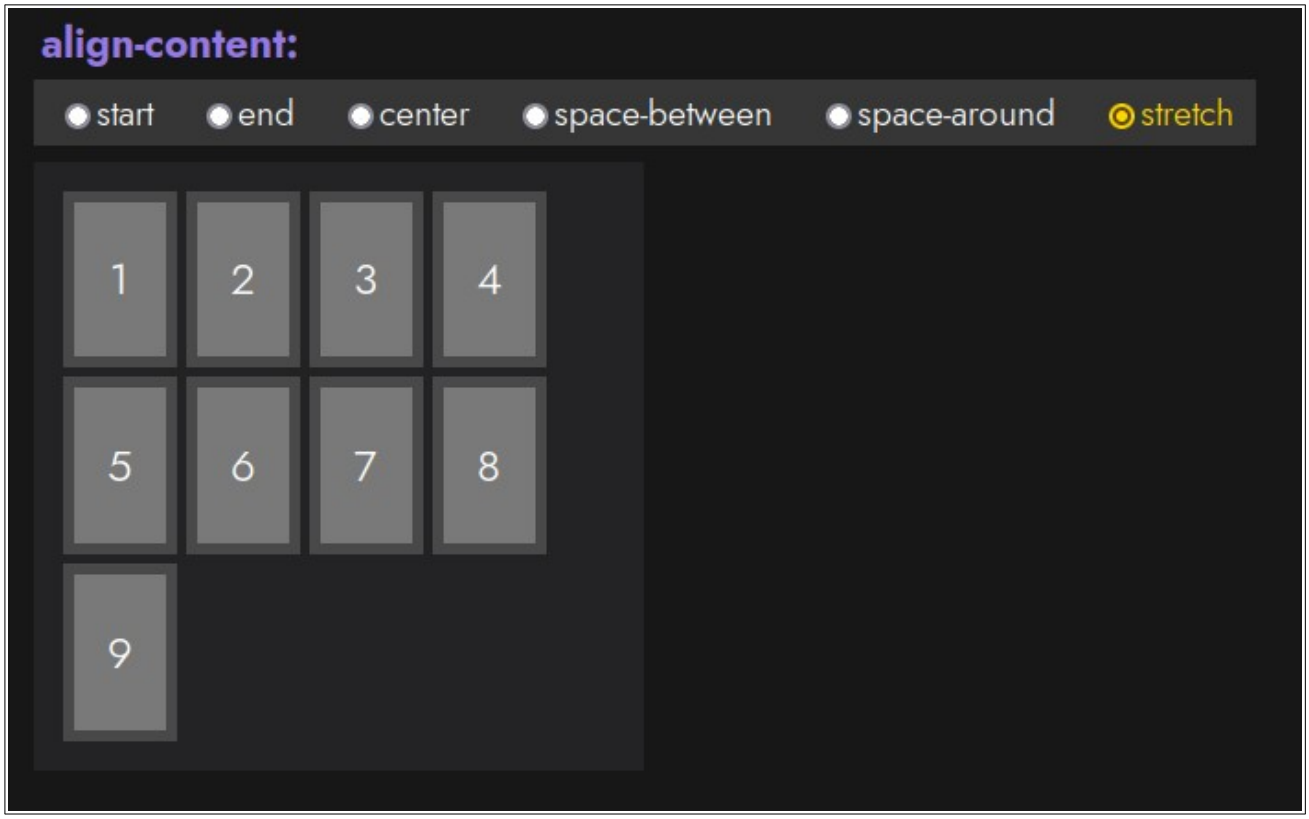

align-content:
start
end
center
space-between
space-around
stretch
1
2
3
4
5
6
7
8
9


align-content:
start
end
center
space-between
space-around
stretch
1
2
3
4
5
6
7
8
9


align-content:
start
end
center
space-between
space-around
stretch
1
2
3
4
5
6
7
8
9


align-content:
start
end
center
space-between
space-around
stretch
1
2
3
4
5
6
7
8
9


align-content:
start
end
center
space-between
space-around
stretch
1
2
3
4
5
6
7
8
9


align-content:
start
end
center
space-between
space-around
stretch
1
2
3
4
5
6
7
8
9

3.7 Alineaciones específicas

Por otro lado, la propiedad `align-self` actúa exactamente igual que `align-items`, sin embargo es la primera propiedad de flex que vemos que se utiliza sobre un **ítem hijo específico** y no sobre el elemento padre contenedor. Salvo por este detalle, funciona exactamente igual que `align-items`.

3.7.1 La propiedad align-self

Observa los valores que puede tomar la propiedad `align-self`:

Propiedad	Valores	Actúa en eje
`align-self`	**auto** | start | end | center | stretch | baseline	2️⃣

Gracias a ese detalle, `align-self` nos permite cambiar el comportamiento de `align-items` y sobreescribirlo con **comportamientos específicos** para ítems concretos que no queremos que se comporten igual que el resto.

La propiedad `align-self` puede tomar los siguientes valores:

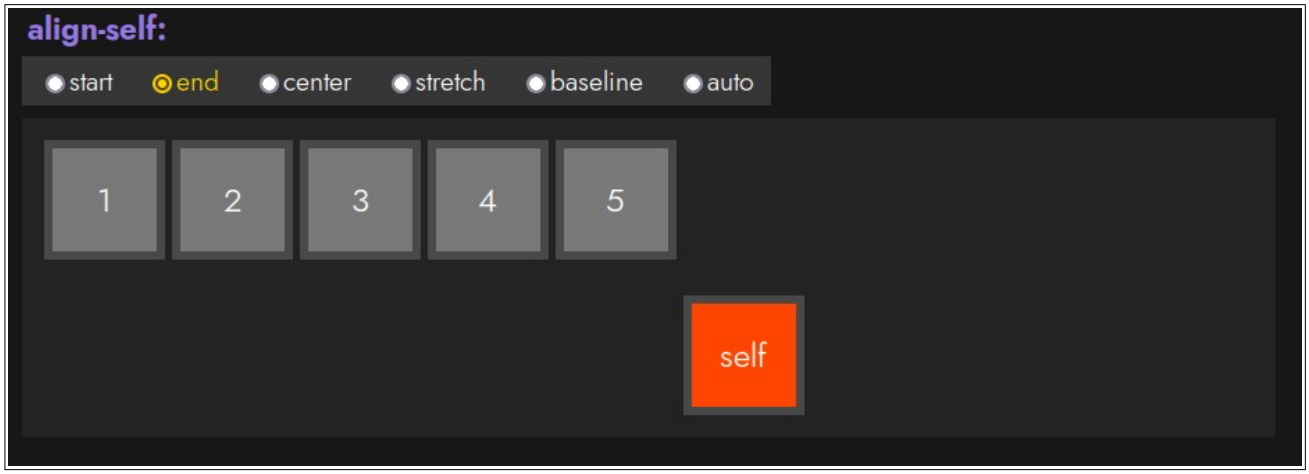
Valor	Descripción
`start`	Alinea los ítems al **inicio** del contenedor.
`end`	Alinea los ítems al **final** del contenedor.
`center`	Alinea los ítems al **centro** del contenedor.
`stretch`	Alinea los ítems **estirándolos** al tamaño del contenedor.
`baseline`	Alinea los ítems en el contenedor según la **base** de los ítems.
auto	Hereda el valor de **align-items** del padre (si no se ha definido, es **stretch**).

ama

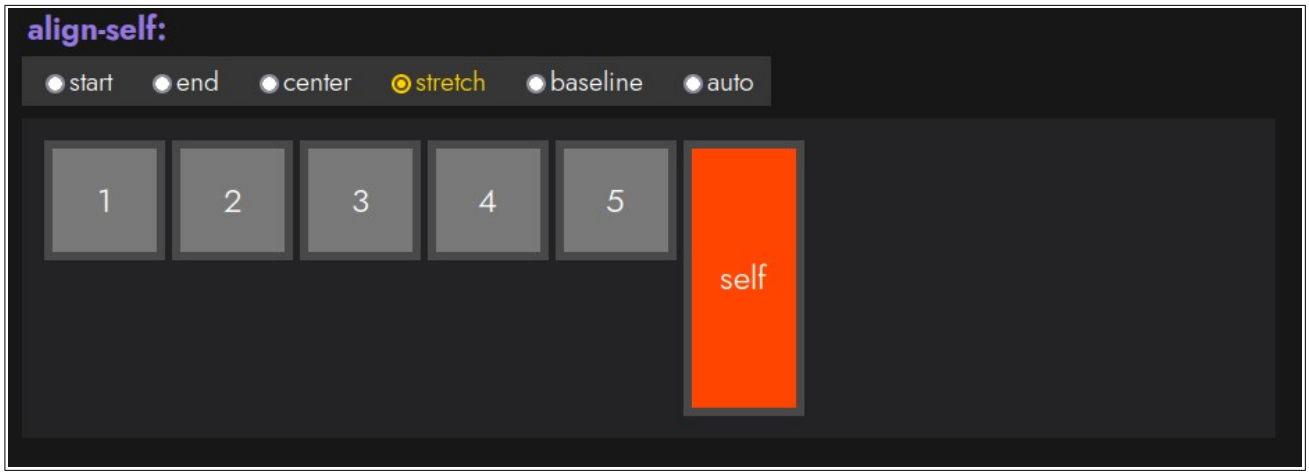
Como vemos, se comporta igual que la propiedad `align-items`, salvo con la adición del valor **auto**, que si se especifica, el navegador le asignará el valor de la propiedad `align-items` del contenedor padre. En caso de no existir, el valor por defecto será **stretch**.

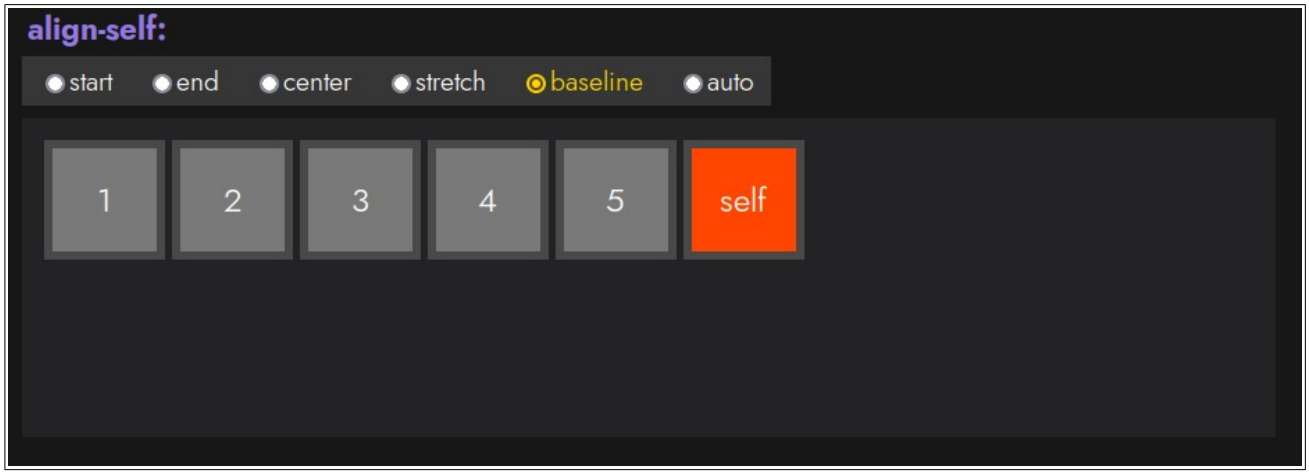
Veamos un ejemplo para verlo en funcionamiento:

align-self:
start
end
center
stretch
baseline
auto
1
2
3
4
5
self


align-self:
start
end
center
stretch
baseline
auto
1
2
3
4
5
self

align-self:
start
end
center
stretch
baseline
auto
1
2
3
4
5
self





3.7.2 Atajo: Alineaciones

Existe una propiedad de atajo con la que se pueden establecer los valores de las propiedades `align-content` y `justify-content` de una sola vez. Dicha propiedad es `place-content` y funciona de la siguiente forma:

```
.container {
  display: flex;

  /* 2 parámetros */
  place-content: start end;
  /* Equivalente a... */
  align-content: start;
  justify-content: end;

  /* 1 parámetro */
  place-content: start;
  /* Equivalente a... */
  align-content: start;
  justify-content: start;
}
```

3.8 Orden de los elementos

Por último, y quizás una de las propiedades más interesantes, es `order`. Se trata de una propiedad mediante la cual podemos modificar y establecer un orden de los elementos mediante números:

Propiedad	Valor	Descripción
`order`	**0** | NUMBER	Número (peso) que indica el orden de aparición de los ítems.

Por defecto, todos los elementos hijos de un contenedor flex tienen establecido un `order` por defecto al valor `0`, aunque no se especifique de forma explícita. Si indicamos una propiedad `order` con un valor numérico diferente, irá recolocando los ítems según dicho número, colocando antes los elementos con un número `order` más pequeño (*incluso valores negativos*) y después los elementos con números más altos.

4. ELEMENTOS FLEXIBLES EN CSS

4.1 Introducción

Como su propio nombre indica, una de las características más interesantes de **flex** es la **flexibilidad** de sus elementos. Existen una serie de propiedades muy interesantes para dotar de interacción y flexibilidad a los elementos que forman parte de un contenedor **flex**.

A excepción de la propiedad `align-self` vista anteriormente, todas las propiedades que hemos visto hasta ahora se aplican sobre el elemento **contenedor**. Las siguientes propiedades, sin embargo, se aplican **sobre los elementos hijos**.

4.2 Propiedades de flexibilidad

Las siguientes propiedades nos permiten dotar de flexibilidad a los elementos hijos de un contenedor **flex**:

Propiedad	Valor	Descripción
`flex-basis`	SIZE | **content**	**Tamaño base** de los ítems antes de aplicar una variación.
`flex-grow`	**0** | NUMBER	Número que indica el factor de **crecimiento** del ítem (si no ocupa el 100% del padre).
`flex-shrink`	**1** | NUMBER	Número que indica el factor de **decrecimiento** del ítem (si ocupa el 100% del padre).

Vamos a ver detalladamente cada una de estas propiedades para entenderlas mejor.

4.2.1 La propiedad flex-basis

En primer lugar, tenemos la propiedad `flex-basis`, que define el **tamaño base** por

defecto que tendrán los ítems antes de aplicarle una cierta distribución de espacio. Se suele aplicar un tamaño específico (*unidades, porcentajes, etc...*), pero también se puede aplicar la palabra clave `content` que ajusta automáticamente el tamaño al contenido del elemento. Este es el valor por defecto de la propiedad.

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  background: grey;
  height: 150px;
  gap: 20px;
}

.item {
  flex-basis: 200px;
  background: red;
}
```

Un detalle interesante sobre esta propiedad, es que si le aplicamos un `flex-direction: column` y aumentamos el `height`, invertiremos los ejes primario y secundario, y `flex-basis` actuará como `height` en lugar de como `width`. Observa también que al estar usando la propiedad de atajo `gap`, el hueco se aplica a ambos ejes, por lo que también actúa.

4.2.2 La propiedad flex-grow

La propiedad `flex-grow` actúa en situaciones donde:

- Hay un `flex-basis` definido.
- Los ítems cubren el tamaño total del contenedor flex padre.

En esas situaciones, la propiedad `flex-grow` indica el factor de crecimiento de los ítems en el caso de que no tengan un ancho o alto específico. Modifiquemos el ejemplo anterior y añadamos el siguiente fragmento de código:

```
.item:nth-child(1) { flex-grow: 1; }
```

Por omisión, todos los elementos tienen un `flew-grow: 0` definido, de modo que no hay factor de crecimiento en el elemento, y tendrá el tamaño definido por la propiedad `flex-basis` que explicamos anteriormente. Sin embargo, si colocamos un `flex-grow: 1`

al primer hijo (*ejemplo anterior*), este crecerá hasta que la suma de los hijos ocupen el 100% del contenedor, mientras que el resto de hijos tendrá el tamaño base definido por `flex-basis`.

Volvamos a modificarlo. Ahora añadiremos las dos últimas líneas de CSS:

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  background: grey;
  height: 150px;
  gap: 20px;
}

.item {
  flex-basis: 200px;
  background: red;
}

.item:nth-child(1) { flex-grow: 1; }
.item:nth-child(2) { flex-grow: 2; }
```

En este caso, el primer elemento tiene un **factor de crecimiento** de `1`, mientras el segundo elemento tiene un **factor de crecimiento** de 2, el resto de elementos tendrán un **factor de crecimiento** de `0`, o sea, no crecen. Así pues, tendremos que el elemento número 3 tiene un tamaño igual al `flex-basis`, mientras que el elemento número 1 crece un poco más y el elemento número 2 crece un poco más que el número 1.

En resumen, `flex-grow` establece un factor de crecimiento observando en conjunto el resto de elementos, y veremos que actúa siempre y cuando la suma del espacio de los elementos hijos no superen el 100% del contenedor padre.

4.2.3 La propiedad flex-shrink

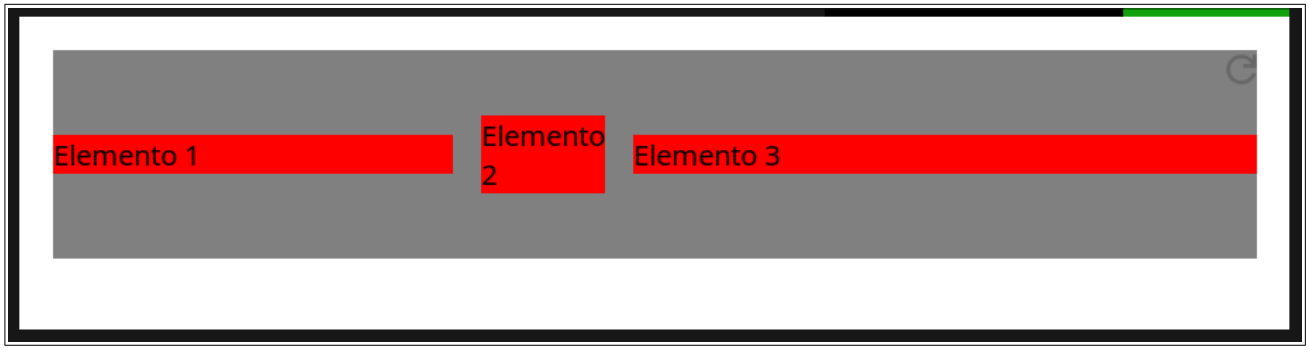
Por otro lado, tenemos la propiedad `flex-shrink` que es la opuesta a la propiedad `flex-grow`. Mientras que la anterior indica un factor de crecimiento, `flex-shrink` hace justo lo contrario: aplica un **factor de decrecimiento**.

La propiedad `flex-shrink` actúa en situaciones donde:

- Hay un `flex-basis` definido.
- Los ítems **no** cubren el tamaño total del contenedor flex padre.

Volvamos al ejemplo inicial de `flex-basis` y vamos a añadir las dos últimas líneas y a cambiar el `flex-basis` de `.item`:

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  background: grey;
  height: 150px;
  gap: 20px;
}

.item {
  flex-basis: 600px;
  background: red;
}

.item:nth-child(1) { flex-shrink: 2; }
.item:nth-child(2) { flex-shrink: 6; }
```



Observa que ahora, hemos definido un `flex-basis` más grande, para obligar que

ocupe más del 100% del contenedor flex padre. Si al primer elemento le aplicamos un `flex-shrink: 2` y al segundo elemento un `flex-shrink: 6`, veremos que el primer elemento va a ser ligeramente más pequeño que el tercero (*que no cambia respecto al tamaño base*), mientras que el segundo elemento va a ser considerablemente más pequeño.

4.2.4 Atajo: La propiedad flex

Existe una propiedad denominada `flex` que sirve de atajo para estas tres propiedades indicadas en los ítems hijos. Veamos como funciona, ya que se trata de una propiedad muy versátil que soporta varios formatos:

Propiedad	Valor	Descripción
`flex` (1 parámetro)	`None` | BASICS | GROW	Establece un tamaño base o un factor de crecimiento.
`flex` (2 parámetros)	GROW | SHRINK	Establece un factor de crecimiento y un factor de decrecimiento.
`flex` (3 parámetros)	**0 1 auto** | GROW | SHRINK | BASICS	Establece el factor de crecimiento y decrecimiento y el tamaño base.

Dependiendo de la cantidad de parámetros:

- Si en la propiedad `flex` indicamos **un sólo parámetro** podemos indicar la palabra clave `none`, que es equivalente a `0 0 auto`, podemos establecer un `flex-grow` si lo que escribimos es un NUMER o en otro caso, podemos escribir un SIZE y definir un `flex-basis`.
- Si en la propiedad `flex` indicamos **dos parámetros** estaremos estableciendo un factor de crecimiento y luego, el factor de decrecimiento.
- Si en la propiedad `flex` indicamos **tres parámetros** estaremos estableciendo los dos factores de crecimiento, y además un tamaño base.

Por defecto, el valor por omisión de esta propiedad es `0 1 auto`.

```
.item {
 /* flex: <flex-grow> <flex-shrink> <flex-basis> */
 flex: 1 3 35%;

 /* Equivalente a... */
 flex-grow: 1;
 flex-shrink: 3;
 flex-basis: 35%;
```

```
}
```

4.3 Elementos flex animados

Ten en cuenta que las propiedades `flex-grow`, `flex-shrink` o incluso su atajo `flex` son animables, por lo que podemos conseguir hermosos efectos de animación con fragmentos de código con muy poco código como el siguiente:

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 150px;
  gap: 5px;
  background: grey;
}

.item {
  flex: 0 1 15%;
  padding: 20px;
  background: black;
  color: white;
  transition: all 0.5s;
}

.item:hover {
  flex: 1 1 10%;
  background: indigo;
}
```

Observa que el truco está en utilizar la propiedad `transition`, que veremos en el apartado de Transiciones, en todos los elementos hijos para preparar una animación cada vez que hagamos el `:hover` y movamos el ratón sobre un elemento que cambia sus valores de flexibilidad mediante `flex` y su color de fondo mediante `background`.

4.4 Transiciones

Las **transiciones CSS** son un tipo de animación muy simple, donde se establecen la forma en la que va a cambiar una o varias propiedades de CSS. Por defecto, en CSS los cambios se producen de forma inmediata y abrupta, por lo que, por ejemplo, un cambio de color a otro se cambiaría de forma instantánea. Utilizando transiciones de CSS podríamos suavizar los cambios, de modo que se verían poco a poco, de forma más sutil.

Por lo general, estas transiciones suelen producirse cuando el usuario realiza una cierta acción. Una de las acciones más básicas: cuando el usuario mueve el ratón sobre un elemento, utilizando el selector (*pseudoclase*) `:hover`.

Al mover el ratón encima de un elemento para ver un cambio de estado se activará el pseudoselector hover. Si no añadimos nada el cambio de un fondo rojo a azúl se producirá **sin transición** de forma instantánea:

```
.element {
  width: 100px;
  height: 100px;
  background: red;
}

.element:hover {
  background: blue;
}
```

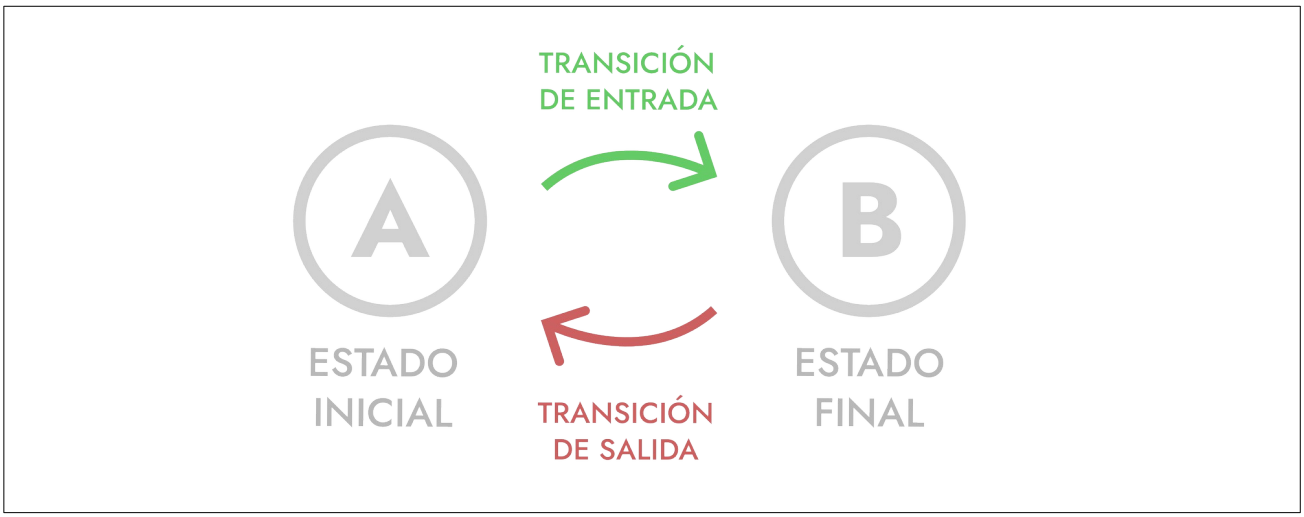
Sin embargo podemos añadir un tiempo de cambio progresivo entre un colo y otro añadiendo la propiedad **transition**: y el tiempo que queremos que tarde entre los dos colores **2s**;

```
.element {
  width: 100px;
  height: 100px;
  background: red;
  transition: 2s;
}

.element:hover {
```

```
  background: blue;
}
```

4.4.1 Transición de entrada y salida

Es interesante tener en cuenta que hay dos tipos de transiciones: de **entrada** y de **salida**. Las transiciones de entrada se aplican cuando el estado inicial cambia al estado final (*se produce la condición: el ratón encima del elemento*), mientras que las transiciones de salida se aplican cuando se deja de cumplir dicha condición (*se quita el ratón encima del elemento*), volviendo a su estado inicial:



Aprovechando los **selectores CSS**, podemos hacer transiciones de entrada y salida diferentes. Por lo general, la transición CSS se coloca en el elemento (*como hicimos en el ejemplo anterior*). En ese caso, aplicaría siempre.

En nuestro ejemplo el estado A serie el elemento y el estado B cuando estamos encima del elemento (hacemos hover).

- Si colocamos transition en A se ejecutará tanto al pasar a B como al salir de B.
- Si colocamos transition en B se ejecutará al pasar a B **pero NO** al salir de B.
- Si colocamos transition en A y en B se ejecutará tanto al pasar a B como al salir de B, pero la transición durará el doble 2s + 2s = 4s.

4.4.2 Transiciones por propiedad

Es posible que queramos indicar (*por ejemplo*) diferentes duraciones dependiendo de la propiedad CSS. Por ejemplo, que tarde mucho en cambiar el ancho del elemento, pero muy poco en cambiar el color de fondo.

Para ello, podemos separar con comas las diferentes propiedades de las que queremos hacer la transición:

```
.element {
  width: 200px;
  height: 200px;
  background: grey;
  transition:
    width 4s,
    background 1s;
}

.element:hover {
  width: 400px;
  background: deeppink;
}
```

5. BIBLIOGRAFÍA

[1] Manz.dev
https://lenguajecss.com/css/flex/que-es-flex/
[2] Manz.dev transiciones
https://lenguajecss.com/animaciones/transiciones/que-son/